



PCI Transaction Level FSDB Dumping

Concept	1
What is a transaction?	1
Transaction support.....	2
Design Unit.....	2
How the PLI Works.....	4
PLI – Background	4
Procedures for Writing a PLI Routine	4
List of sub-function C Files for corresponding FSDB writer API	7
General Steps for Writing FSDB	8
Open an FSDB.....	8
Set Scale Unit and Other Information.....	8
Choose Tree Creation Scheme	9
Initialize Data Type Creation (if necessary)	10
Create the Design Hierarchy	10
Create the Value Changes	11
Close the FSDB	12
Transaction Recording	13
Transactions Creation Steps	13
Dumping Transactions in FSDB Steps	13
Practice.....	15
Revision History	16

Concept

Novas Software wrote a "simple" simulation-time transaction recognizer for Cisco/RTP for the PCI bus protocol. Novas Software provides the following new PLI tasks to dump transactions along with normal HDL signals to a single FSDB file during simulation:

- \$fsdb_tr_file
- \$fsdb_tr_stream
- \$fsdb_tr_attribute
- \$fsdb_tr_begin
- \$fsdb_tr_data
- \$fsdb_tr_abort
- \$fsdb_tr_end
- \$fsdb_tr_close

What is a transaction?

- Transactions are an important piece of abstraction in system design and debug. System design is at a very early stage of the whole design process; therefore, a powerful viewing mechanism for transactions is mandatory to system designers.



- A transaction is composed of streams and a transaction. The transaction can be dumped into a transaction FSDB file using APIs named Open Transactor Interface or OTI¹.
- Streams hold transactions. Each transaction consists of a set of attributes and is independent of one another. That is, there is no such concept as "transaction type", as in SCV, even if the sets of attributes that constitute two transactions are the same.
- When you create a transaction, you must follow the steps below:
 1. Create a stream.
 2. Create attributes.
 3. Create transaction.

Transaction support

The current support for transaction level verification and debug in Debussy/Verdi 5.4 is as follows:

- Transaction waveform visualization in *nWave*.²
- Spreadsheet view that supports the sorting/filtering of transactions.
- FSDB transaction dumping from SystemC/SCV.³
- Updated APIs to allow partners to dump transactions in FSDB format.⁴

Design Unit

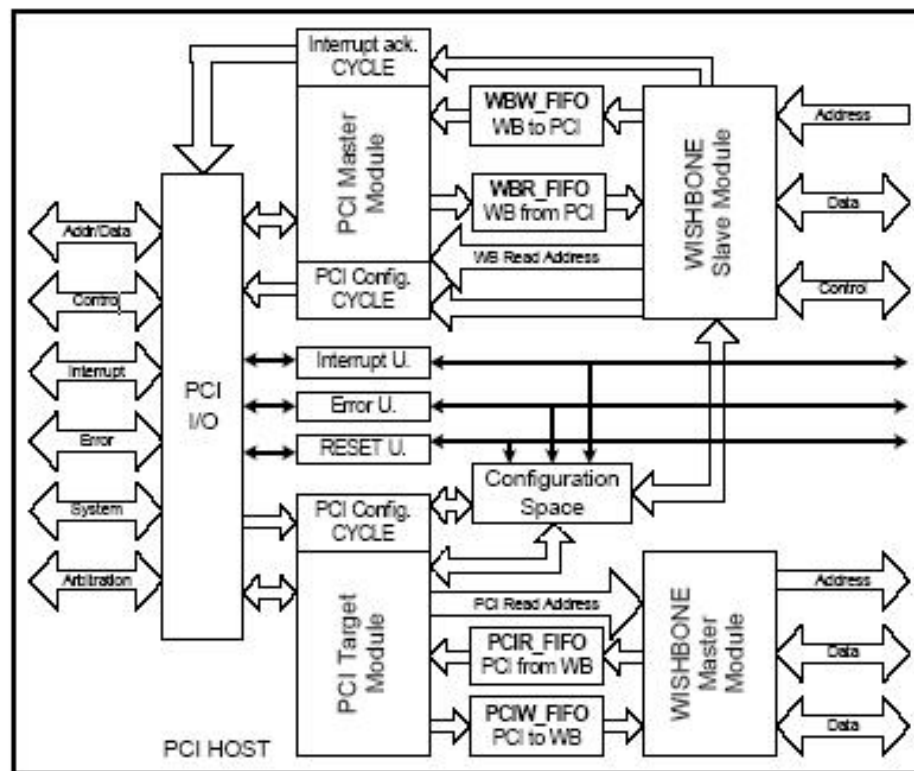
An open source code design "PCI Bridge IP Core" from <http://www.opencores.org> was used. The block diagram is shown in [Figure 1](#).

¹ This is simply an extension of what we have done in the past with users who dump signal data to FSDB from their proprietary applications/simulators/models.

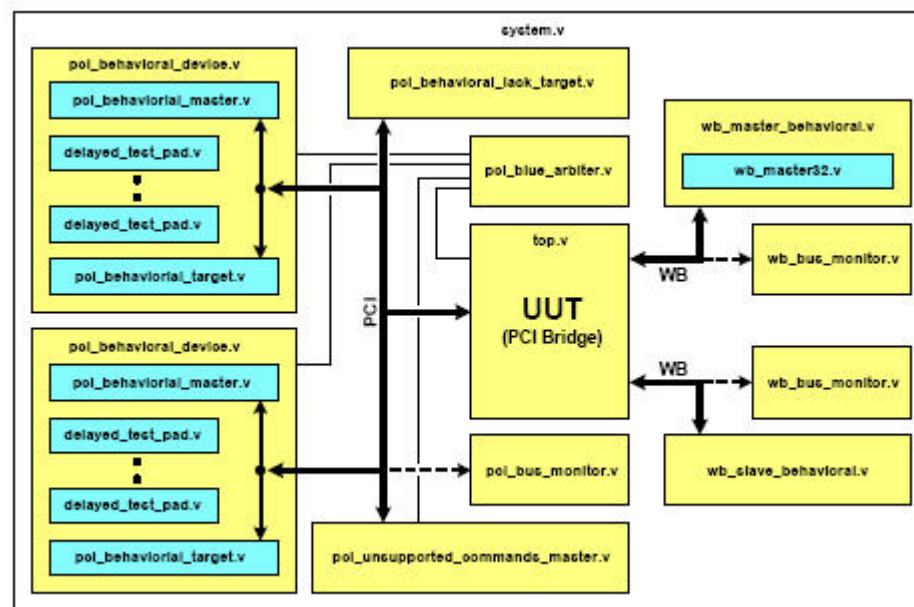
² We have posted an application note "Viewing Transactions in nWave"

³ Novas provide 2 libraries to support the signals and transactions dumping in SystemC, One library: libfsdbSC.a that supports dumping SystemC design in an FSDB file. Another library is scv_tr_fsdb.a that supports the SCV transaction recording API that can dump the transactions to an FSDB file. Reference material: <Novas_install_path>/doc/linking.pdf, Linking with SystemC Simulators session.

⁴ We are actively working with partners to support transaction dumping from commercially-available IP. Some of these packages (Denali/PCI-Express, Spiritech/AMBA) are available today.


Figure 1: **CORE ARCHITECTURE OVERVIEW**

The Testbench Module Hierarchy is shown in [Figure 2](#). All new system tasks were inserted in module pci_bus_monitor (package\bench\verilog\pci_bus_monitor.v file).


Figure 2: **TESTBENCH MODULE HIERARCHY**



How the PLI Works

PLI – Background

- Write C functions that have PLI routines.
- Use veriuser.c to associate the C function with the simulator system task.
- Compile veriuser.c and C functions dynamically to generate shared lib (*.DLL in Windows and *.so in UNIX). A simulator like NCSIM allows dynamic linking.
- Based on simulator, pass the C/C++ function details to simulator during the compile process of Verilog code. (This is called linking, and you need to refer to simulator user guide to understand how this is done.)
- Once linked, just run the simulator like any normal Verilog simulation.
- During execution of the Verilog code by the simulator, when the simulator encounters the user defined system tasks (those start with \$), the execution control is passed to the PLI routine (C/C++ function).

Procedures for Writing a PLI Routine ⁵

1: Include the header files

The file veriuser.c containing the main PLI/C program must have

```
#include <veriuser.h>
#include <vxl_veriuser.h>
```

2: Function prototype and variable declaration

This part of the program contains all local variables and the functions that it invokes as part of the system call. In the present case, as explained in the next step, it will be as shown below.

```
int my_calltf(), my_checktf();
```

However, if the functions are in separate files, they should be declared as external functions.

```
extern int plicompile_tr_file();
extern int plitask_tr_file();
extern int plimisc_tr_file();

extern int plicompile_tr_scope();
extern int plitask_tr_scope();
extern int plimisc_tr_scope();

extern int plicompile_tr_stream();
extern int plitask_tr_stream();
extern int plimisc_tr_stream();

extern int plicompile_tr_attribute();
extern int plitask_tr_attribute();
```

⁵ Please refer OpenFsdbWriter.pdf for detail too.

```

extern int plimisc_tr_attribute();

extern int plicompile_tr_begin();
extern int plitask_tr_begin();
extern int plimisc_tr_begin();

extern int plicompile_tr_data();
extern int plitask_tr_data();
extern int plimisc_tr_data();

extern int plicompile_tr_abort();
extern int plitask_tr_abort();
extern int plimisc_tr_abort();

extern int plicompile_tr_end();
extern int plitask_tr_end();
extern int plimisc_tr_end();

extern int plicompile_tr_close();
extern int plitask_tr_close();
extern int plimisc_tr_close();

```

3: Create essential data structure

There are a number of data structures that must be defined in a PLI program. These are the variables through which Verilog communicates with the C code. Veriuserdfs[]: the main interaction between the C-code that one writes and the Verilog simulator is done through a table. The Verilog simulator looks at this table and figures out the properties the system call corresponding to this PLI routine would be associated with.

```

static s_tfcell deb_veriuserdfs[] =
#else
s_tfcell veriuserdfs[] =
#endif
{
    {
        usertask,          /* type of PLI routine */
        0,                /* user_data value */
        plicompile_tr_file, /* checktf routine */
        0,                /* sizetf routine */
        plitask_tr_file,   /* calltf routine */
        plimisc_tr_file,   /* misctf routine */
        "$fsdb_tr_file",   /* system task/function name */
    },
    {
        usertask,
        0,
        plicompile_tr_stream,
        0,
        plitask_tr_stream,
        plimisc_tr_stream,
        "$fsdb_tr_stream",
    },
}

```



```
1
},
{
  usertask,
  0,
  plicompile_tr_attribute,
  0,
  plitask_tr_attribute,
  plimisc_tr_attribute,
  "$fsdb_tr_attribute",
  1
},
{
  usertask,
  0,
  plicompile_tr_scope,
  0,
  plitask_tr_scope,
  plimisc_tr_scope,
  "$fsdb_tr_scope",
  1
},
{
  usertask,
  0,
  plicompile_tr_begin,
  0,
  plitask_tr_begin,
  plimisc_tr_begin,
  "$fsdb_tr_begin",
  1
},
{
  usertask,
  0,
  plicompile_tr_data,
  0,
  plitask_tr_data,
  plimisc_tr_data,
  "$fsdb_tr_data",
  1
},
{
  usertask,
  0,
  plicompile_tr_abort,
  0,
  plitask_tr_abort,
  plimisc_tr_abort,
  "$fsdb_tr_abort",
  1
},
{
```

```

    usertask,
    0,
    plicompile_tr_end,
    0,
    plitask_tr_end,
    plimisc_tr_end,
    "$fsdb_tr_end",
    1
},
{
    usertask,
    0,
    plicompile_tr_close,
    0,
    plitask_tr_close,
    plimisc_tr_close,
    "$fsdb_tr_close",
    1
},
{0} /** final entry must be 0 ***/
};

```

4: tf routines

- checktf routine
 - o Optional
 - o Simulator checks the routine once right before simulation
- calltf routine
 - o Perform the task or function
- sizetf
 - o Returns the size
 - o Has to be userfunction or userRealfunction routine
 - o Default returns 32-bit values
- misctf
 - o Routine called depending upon reasons e.g. reason_endofcompile, reason_paramvc etc.

List of sub-function C Files for corresponding FSDB writer API

fsdb_tr_abort.c	Abort transaction
fsdb_tr_attribute.c	Create transaction attribute name
fsdb_tr_begin.c	Begin address transaction and Create Value change
fsdb_tr_close.c	Close the FSDB file
fsdb_tr_data.c	Begin data transaction and Create Value change
fsdb_tr_end.c	End Transaction
fsdb_tr_file.c	Create transaction file name
fsdb_tr_scope.c	Get transaction hierarchical scope name from user to create the FSDB Signal Tree
fsdb_tr_stream.c	Create transaction stream name in FSDB

General Steps for Writing FSDB

- Open an FSDB file
- Set scale unit and other information
- Choose tree creation scheme
- Initialize data type creation if necessary
- Create the design hierarchy
- Create the value changes
- Close the FSDB file

Open an FSDB

An FSDB is opened by `ffw_Open` API, which asks the application to give the file name and file type. The file type tells the source of the information, and is defined as `fsdbFileType` in `fsdbShr.h` header file. Once the file is opened successfully, the application can get a pointer to the FSDB writer object, which is a must parameter for most FSDB Writer APIs. See [Figure 3](#)

```
//-----
//
// Function call for __OpenFsdbFileForWrite
//
//-----

static void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[], fsdbFileType file_type)
{
    *fsdb_obj = ffw_Open(fname, file_type);

    if (NULL == *fsdb_obj) {
        printf("Fail to open a fast file object! File name = %s.\n", fname);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    printf("=====\n");
    printf("Creat FSDB transaction file %s !\n", fname);
    printf("=====\n");

    ffw_CreateTreeByHandleScheme(*fsdb_obj); // Tree Creation

    ffw_SetScaleUnit(*fsdb_obj, "1n");
}
```

Figure 3: *OPEN AN FSDB FILE IN FSDB_TR_FILE.C*

Set Scale Unit and Other Information

See Figure 4 for the APIs to set scale unit.

```
//-----
//
// Function call for __OpenFsdbFileForWrite
//
//-----

static void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[], fsdbFileType file_type)
{
    *fsdb_obj = ffw_Open(fname, file_type);

    if (NULL == *fsdb_obj) {
        printf("Fail to open a fast file object! File name = %s.\n", fname);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    printf("=====\n");
    printf("Creat FSDB transaction file %s !\n", fname);
    printf("=====\n");

    ffw_CreateTreeByHandleScheme(*fsdb_obj); // Tree Creation
    ffw_SetScaleUnit(*fsdb_obj, "1n");
}
```

Figure 4: **SET SCALE UNIT AND OTHER INFORMATION**

If the simulation does have a scale unit, then the application must set it. If it does not, then the default value is 1ns. Other information such as simulation date and simulator version are optional and for reference only.

Choose Tree Creation Scheme

See [Figure 5](#)

```
//-----
//
// Function call for __OpenFsdbFileForWrite
//
//-----

static void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[], fsdbFileType file_type)
{
    *fsdb_obj = ffw_Open(fname, file_type);

    if (NULL == *fsdb_obj) {
        printf("Fail to open a fast file object! File name = %s.\n", fname);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    printf("=====\n");
    printf("Creat FSDB transaction file %s !\n", fname);
    printf("=====\n");

    ffw_CreateTreeByHandleScheme(*fsdb_obj); // Tree Creation
    ffw_SetScaleUnit(*fsdb_obj, "1n");
}
```

Figure 5: **CHOOSE TREE CREATION SCHEME**

The tree creation scheme is chosen by calling `ffw_CreateTreeByIdcodeScheme` or `ffw_CreateTreeByHandleScheme` API. The default is idcode scheme if nothing is called.

Initialize Data Type Creation (if necessary)

If there is user defined data type, then the application must call the data type creation API to notify the FSDB writer, so that it can initialize the necessary data structures to store the data type definition. This can be done by calling the `ffw_GetDataTypeCreationReady` API. For most cases, there aren't user defined data types. Therefore, by default, the FSDB Writer assumes there are no user defined data types. See [Figure 6](#)

```

// Function call for __CreateScope
static void
__CreateScope(FfwObject* ffw_obj)
{
    extern int Stream_Created_Count;
    extern char my_stream[100][100];
    extern int Attribute_Created_Count;
    extern char my_attribute[100][100];
    extern char str_value[100][256];
    extern char mod_value[2560];

    char *scope_ptr;
    char scope_value[256];
    int num, num1;

    if (!Tree_Created_Flag) {
        ffw_GetDataTypeCreationReady(ffw_obj);
        ffw_BeginTree(ffw_obj);
        ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, strtok(mod_value, ","));
        while ((scope_ptr = strtok(NULL, ",")) != NULL) {
            strcpy(scope_value, scope_ptr);
            ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, scope_value);
        }
        for (num = 0; num < Stream_Created_Count; num++) {
            stream_hdl[num] = ffw_CreateStream(ffw_obj, my_stream[num]);
            printf("Create Stream name = %s\n", my_stream[num]);
        }
        for (num1 = 0; num1 < Attribute_Created_Count; num1++) {
            attr_hdl[num1] = ffw_CreateAttr(ffw_obj, my_attribute[num1], FSDB_ATTR_DT_CHAR, 0, 8, FALSE);
            printf("Create Attribute name = %s\n", my_attribute[num1]);
        }
        ffw_EndTree(ffw_obj);
        Tree_Created_Flag = 1;
    }
}

```

Figure 6: **INITIALIZE DATA TYPE CREATION (IF NECESSARY)**

Create the Design Hierarchy

The design hierarchy is composed by calling tree creation APIs. The `ffw_BeginTree` API creates a top scope, which has no name. The `ffw_CreateScope` API creates a scope, which is a child of "the current scope," and then moves "the current scope" down to the newly created one. The `ffw_CreateUpscope` moves "the current scope" up to its parent scope. The `ffw_CreateVarByIdcode` or `ffw_CreateVarByHandle` API creates a variable, which belongs to "the current scope." The `ffw_EndTree` API completes the design hierarchy. Note that an FSDB file may contain none, one, or multiple design hierarchies. See [Figure 7](#)

```

//
// Function call for __CreateScope
//
static void
__CreateScope(FfwObject* ffw_obj)
{
    extern int Stream_Created_Counter;
    extern char my_stream[100][100];
    extern int Attribute_Created_Counter;
    extern char my_attribute[100][100];
    extern char str_value[100][256];
    extern char mod_value[2560];

    char *scope_ptr;
    char scope_value[256];
    int num, num1;

    if (!Tree_Created_Flag) {
        ffw_GetDataTimeCreationReady(ffw_obj);
        ffw_BeginTree(ffw_obj);
        ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, strtok(mod_value, "."));
        while ((scope_ptr = strtok(NULL, ".")) != NULL) {
            strcpy(scope_value, scope_ptr);
            ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, scope_value);
        }
        for (num = 0; num < Stream_Created_Counter; num++) {
            stream_hdl[num] = ffw_CreateStream(ffw_obj, my_stream[num]);
            printf("Create Stream name = %s\n", my_stream[num]);
        }
        for (num1 = 0; num1 < Attribute_Created_Counter; num1++) {
            attr_hdl[num1] = ffw_CreateAttr(ffw_obj, my_attribute[num1], FSDB_ATTR_DT_CHAR, 0, 8, FALSE);
            printf("Create Attribute name = %s\n", my_attribute[num1]);
        }
        ffw_EndTree(ffw_obj);
        Tree_Created_Flag = 1;
    }
}

```

Figure 7: CREATE THE DESIGN HIERARCHY IN FSDB_TR_SCOPE.C

Create the Value Changes

Conceptually, a value change is composed of a pair: a time and a value. The time is created by calling `ffw_CreateXCoorByHnL`, while the value is created by calling `ffw_CreateVarValueByIdcode` or `ffw_CreateVarValueByHandle` API. See [Figure 8](#)

```

//
// Function call for __CreateVC
//
fsdbXTag      run_time;

void
__CreateVC(FfwObject* ffw_obj, int current_time, char* trans_id_str, char* address_str) {
    fsdbTag64      xtag;
    str_T          ret_ffw_str;

    char* cfg_rd_str;
    char* cfg_wr_str;
    char* io_rd_str;
    char* io_wr_str;
    char* mem_rd_str;
    char* mem_wr_str;
    char* master_abort_str;
    char* idle_str;

    fsdbAttrHdlVal attr_val[2];

    xtag.H = 0;
    xtag.L = current_time;

    run_time.hitag.H = 0;
    run_time.hitag.L = current_time;

    ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);

    attr_val[0].hdl = attr_hdl[0];
    attr_val[0].value = (byte_T*) address_str;

    attr_val[1].hdl = attr_hdl[1];
    attr_val[1].value = (byte_T*) "_____";
}

```

Figure 8: CREATE THE VALUE CHANGES

Close the FSDB

An FSDB is closed by `ffw_Close`, which flushes the necessary in-core data and temporary files to the FSDB file. Then it performs some clean up tasks and completes the FSDB file. See [Figure 9](#).

```
//  
#include "stdio.h"  
#include "stdlib.h"  
#include "assert.h"  
#include "ffwAPI.h"  
  
char* fsdb_tr_file_name;  
ffwObject* ffw_obj;  
  
//-----  
//  
// Function call for fsdb_tr_close  
//  
//-----  
  
void  
fsdb_tr_close(void){  
    ffw_Close(ffw_obj);  
}
```

Figure 9: **CLOSE THE FSDB**

Transaction Recording

Transactions Creation Steps

1. Create a stream `ffw_CreateStream()`
2. Create attributes `ffw_CreateAttr()`
3. Create transaction `ffw_BeginTransaction()`
 `ffw_EndTransaction()`

Dumping Transactions in FSDB Steps

Step 1: Create transaction file name in FSDB.

`$fsdb_tr_file`

Description: To create a transaction file name to the FSDB file.

Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_file} {"FILE_ID"}  
(in Verilog) $fsdb_tr_file("FILE_ID");
```

Examples

```
ncsim> call {$fsdb_tr_file} {"pci_tr.fsdb"}
```

Step 2: Create transaction stream name in FSDB.

`$fsdb_tr_stream`

Description: To create a transaction stream name to the FSDB file.

Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_stream} {"STREAM_ID"}  
(in Verilog) $fsdb_tr_stream("STREAM_ID");
```

Examples

```
ncsim> call {$fsdb_tr_stream} {"CON_RD"}  
ncsim> call {$fsdb_tr_stream} {"CON_WR"}  
ncsim> call {$fsdb_tr_stream} {"IO_RD"}  
ncsim> call {$fsdb_tr_stream} {"IO_WR"}  
:  
:  
:
```

Step 3: Create transaction attribute name in FSDB.

`$fsdb_tr_attribute`

Description: To create a transaction attribute name to the FSDB file.



Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_attribute} {"ATTRIBUTE_ID"}  
(in Verilog) $fsdb_tr_attribute("ATTRIBUTE_ID");
```

Examples

```
ncsim> call {$fsdb_tr_attribute} {"addr"}  
ncsim> call {$fsdb_tr_attribute} {"data"}
```

Step 4: Create transaction hierarchical scope name in FSDB.

\$fsdb_tr_scope

Description: To create a transaction hierarchical scope name in fsdb

Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_scope} {"Top.level1.level2.level3...."}  
(in Verilog) $fsdb_tr_scope("Top.level1.level2.level3.....");
```

Examples

```
ncsim> call {$fsdb_tr_scope} {"SYSTEM.monitor32"}
```

Step 5: Begin transaction in FSDB (for this PCI transaction address phase).

\$fsdb_tr_begin

Description: To begin a transaction for certain trans_id

Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_begin} {"TRANS_ID",address_attribute}  
(in Verilog) $fsdb_tr_begin("TRANS_ID", address_attribute);
```

Examples

```
$fsdb_tr_begin("IO_READ",ad_prev[PCI_BUS_DATA_RANGE:0]);
```

Step 6 : Begin transaction in FSDB (for this PCI transaction data phase).

\$fsdb_tr_data

Description: To begin a data transaction for the previous PCI transaction

Syntaxes

```
(in ncsim.rc) ncsim> call {$fsdb_tr_data} {data_attribute}  
(in Verilog) $fsdb_tr_data(data_attribute);
```

Examples

```
$fsdb_tr_data("PCI_DATA",pci_ext_ad[PCI_BUS_DATA_RANGE:0]);
```

Step 7a: Abort transaction in FSDB (for this PCI transaction Abort phase).

\$fsdb_tr_abort

Description: To abort any previous PCI transaction and add abort transaction.

Syntaxes



```
(in ncsim.rc) ncsim> call {$fsdb_tr_abort}  
(in Verilog) $fsdb_tr_abort;
```

Examples
\$fsdb_tr_abort;

Step 7b : End transaction in FSDB (for this PCI transaction End phase).

\$fsdb_tr_end

Description: To abort any previous PCI transaction including abort transaction

Syntaxes
(in ncsim.rc) ncsim> call {\$fsdb_tr_end}
(in Verilog) \$fsdb_tr_end;

Examples
\$fsdb_tr_end;

Step 8: Close transaction in FSDB.

\$fsdb_tr_close

Description: To close a transaction in the FSDB.

Syntaxes
(in ncsim.rc) ncsim> call {\$fsdb_tr_close}
(in Verilog) \$fsdb_tr_close;

Examples
\$fsdb_tr_close;

Practice

To run the demo example please do the following:

1. cd ./link
make
It will generate the libpli.so and pli.a for the pli linking

2. cd ../sim/rtl_sim/run
make

It will run ncvglog, ncelab, ncsim for the PCI design FSDB dumping



Revision History

<i>Revision</i>	<i>Date</i>	<i>Description</i>
1.0	1/4/2005	Initial release.